Seeing the whole agent harness

Power-user platforms, first principles and UI decisions for Agnostic AI

Power users need to understand how an agent is configured: its instructions, capabilities, persistent knowledge, permissions and relationships. A clear interface connects these parts to their sources and makes changes inspectable.

Main finding. The strongest precedent is an inspectable agent setup beside agent-assisted changes. A useful Agnostic AI UI should explain what exists, where it comes from, where it applies and what changed. It should preserve canonical files as the source of truth.

Three particularly useful references

Dify separates agent-assisted building from previewing behavior. Letta makes persistent memory an explicit, versioned resource. LangSmith Fleet distinguishes persistent knowledge, human approvals and the identity under which tools act. [1–4]

Recommended information architecture

Keep the workspace as the top level, then let users inspect an agent or reusable role and its attached resources. Treat the six harness facets as a cross-cutting explanation, rather than forcing every feature into six separate navigation silos.

Core product questions

What makes up this setup? Which configuration applies to each target? What persists between tasks? What changed? What evidence shows that a run used the intended configuration?

The product combines a workspace overview, agent profiles, resource inspectors, configuration comparisons and optional execution evidence. Canonical files remain the source of truth.

The closest configuration precedents

Platform / focus	Documented interaction	What transfers
Dify · new Agent beta [1]	Configure groups model, prompt, skills, files, tools and environment. Build chat edits a visible draft; Apply/Discard reviews changes. Preview is separate.	Agent changes setup; human can inspect the resulting structure and change set.
Letta · persistent agents [2]	MemFS represents agent memory as a Git-backed file hierarchy. Root files enter context; deeper indexed content can be retrieved as needed.	Show the relationship between stored knowledge and selected context.
LangSmith Fleet [3,4]	Agent workspace combines tools, skills and memory. Sharing and execution credentials are separate; an inbox collects approval requests.	Agent profile plus persistent resources, identity and human decision points.
CrewAI AMP / Studio [5,6]	Chat and visual editor share state. Connected agents/tasks sit beside resource panels. Execution has its own timeline and logs.	Explain composition separately from evidence of a run; expose configuration layers.
Relevance AI [7]	Workforce canvas connects specialist agents. Discretionary AI handoffs differ from mandatory next-step transitions.	Label the meaning of a relationship; an edge is not just a decorative connection.

Common configuration model

An agent or team is an inspectable object with attached capabilities and resources. Agnostic AI can use this familiar structure while preserving its read-only view across multiple targets and canonical configuration files.

Canvas builders and cloud control planes

Platform	Documented surface	Transfer / boundary
Flowise Agentflow V2 [8]	Executable graph with agent, tool and control nodes; shared flow state and node configuration.	Useful for actual control flow. Do not use its workflow graph as a metaphor for all files.
Langflow [9]	Component canvas with a selected-component inspector and advanced parameters.	Progressive disclosure: summary first, exact configuration on selection.
Google Agent Studio [10]	Agent/subagent flow; node details expose instructions, model and tools. Preview provides execution events.	Graph is meaningful when it represents real delegation. Keep definition and run evidence distinct.
Microsoft Foundry [11]	Development lifecycle distinguishes experimentation, versioned definitions and published agents.	Make working configuration, immutable version and deployed state legible.
Amazon Bedrock [12]	Version inspector shows model, action groups and knowledge bases; version contents are immutable.	An inspected historical version must be unmistakable from an editable draft.

A reference with a lifecycle caveat

AutoGen Studio’s visual/JSON configuration pairing is useful, but AutoGen is now in maintenance mode. Do not base a forward-looking product recommendation on its continuing feature development. [13,14]

Cross-target configuration visibility

Agnostic AI’s interface should connect canonical files, target-specific transformations and observed runtime outcomes. Each relationship needs a source, a timestamp and an explicit status when resolution or runtime evidence is unavailable.

Composition, persistence and change

Dify: inspectable agent-assisted changes

In Build mode the configuration panel is read-only while the agent makes changes. Its file browser distinguishes persistent setup files from temporary work. Restoring a version restores persistent files too. However, DSL export excludes skills/files, and sandbox-installed CLI tools are absent from the Tools list. [1]

Design implication: a tidy inventory is only trustworthy if it states its coverage. Show untracked resources and incomplete exports explicitly. Borrow the visible change review; do not add Apply buttons to a deliberately read-only viewer.

Letta and Fleet: “memory” needs a lifecycle

Letta’s memory files persist as part of an agent’s identity. Fleet distinguishes transient thread files from persistent files under /memories/, including instructions and skills. These are different lifetimes and uses, even when they share a filesystem representation. [2,4]

Design implication: every knowledge item needs scope, lifetime and loading behavior. Distinguish stored, eligible for retrieval, and observed in context. A file existing on disk does not prove that a particular run used it.

CrewAI: there can be more than one source layer

CrewAI documents organization-level policies that override deployment settings for supported policy controls. Studio exports are one-way: editing exported code does not update the Studio project. [5,6]

Design implication: expose the winning value and its source when precedence is known. Mark forks and exports as separate representations; never imply round-trip synchronization unless it exists.

Fleet and Relevance: capability is not authority

Fleet distinguishes who may run or edit an agent from whose credentials it uses. Relevance distinguishes discretionary delegation from mandatory handoff. [3,7]

Design implication: separate “can access this tool,” “is allowed to act,” and “will follow this route.” These answer different operator questions.

Design for understanding and accountable change

Principle	Tactical decision for Agnostic AI
Objects before abstractions	Start with workspace, target and reusable agent/role. Link instructions, skills, tools and knowledge to them. Keep the six facets available as filters and an overview.
One canonical truth	Read canonical sources. For every card provide source path, scope and representation. Generated target config is a projection, with its own timestamp.
Explain effective behavior carefully	Use Meaning / Source / Effective views. Only call a value “resolved” when the target’s precedence is implemented and known; otherwise show unresolved.
Time is part of configuration	Keep source state, generated output and observed run snapshot separate. A change since the last sync or run should be visible.
Persistence is not context	Show lifetime and selection separately. Represent durability through state restoration and continuity.
Authority is explicit	Display execution identity, declared permissions and approval requirements separately. Label enforcement as unknown without evidence.
Relations need semantics	Use graphs for actual delegation, dependencies or scope. Use inventories and tables for complete comparison.
Human review has a place	Link intent, proposed setup changes, review and outcomes around the inner loop. Keep lessons traceable to the source changes they produced.

The cognitive harness is the human-facing outer loop of intent, review and learning. The agent harness is the inner loop of context, state, control, authority, tools and verification.

What looks powerful but misleads

Temptation	Why it fails	Better choice
A canvas of everything	Files, settings and executions have different relations. A global graph falsely implies control flow.	Scoped relationship graphs with named edge types.
Six equal boxes as the whole UI	The conceptual facets overlap across real features; users must guess where to look.	Object navigation, with facets as alternate lenses.
A single “Memory” bucket	Instructions, retained facts, task files and current context have different lifetimes.	Scope + lifetime + loading behavior.
The Tools list is total capability	Capabilities may also arise from scripts, sandbox software and delegated agents. Dify documents this explicitly. [1]	Show inventory coverage and unknown resources.
Configured means active	A declaration may not be generated, loaded, authorized or invoked.	Separate declared, generated and observed states.
A green compliance score	Static config does not prove execution obeyed it.	Specific checks with evidence and freshness.
Draft and live in one view	Users cannot tell what a current run actually uses.	Explicit snapshot/version labels and comparison.
Export implies synchronization	Some exports are incomplete or one-way. [1,5]	Coverage manifest and source-of-truth label.
Chat is the only interface	Conversation obscures current structure and cumulative changes.	Persistent setup inspector beside change history.
Readable equals editable	Adding forms creates a second authoring system and drift risk.	Read-only inspection with canonical source links.

Five views of the agent setup

Screen	Primary question	Contents
Workspace overview	What setup do I have?	Targets and role definitions; grouped feature inventory; source/sync status; six-facet summary. No invented health score.
Agent / role profile	What shapes this agent?	Purpose and instructions; model where known; attached skills, tools and knowledge; scope and dependencies. Distinguish reusable role from persistent running agent.
Resource inspector	Where does this come from?	Meaning / Source / Effective; ownership, scope, precedence and target support. Knowledge adds lifetime and loading behavior.
Changes and versions	What changed, and where?	Semantic diff grouped by real feature; source revision; affected targets; generated output comparison. Human decisions remain traceable.
Evidence drill-down	What actually happened?	Optional runs, tool calls and checks tied to a configuration snapshot. Without telemetry, say “not observed.”

How the two harnesses appear

The outer cognitive loop supplies intent, review and learning. These appear as context around the setup and its changes. The inner loop explains how the agent receives context, retains state, exercises control and authority, uses tools and verifies results. Avoid creating a second duplicate feature inventory for the outer loop.

The first slice to validate

A user selects a workspace and target, inspects a role and its resources, then follows one setting to its source and explains a change. This tests the core comprehension value before adding runtime dashboards or graphical orchestration.

What remains useful as models improve

Likely durable needs	Likely to shrink or become optional
Who owns the setup; whose credentials are used; what is authorized.	Manual prompt wording and model-specific behavior workarounds.
What information persists; what a run could access; retention and provenance.	Hand-authored retrieval and routing rules where models reliably infer them.
Which version and source produced an outcome; what changed.	Large panels of low-level tuning parameters needed only by weaker models.
Human intent, acceptance criteria, review and accountability.	Rigid task decomposition and unnecessary specialist-agent scaffolding.

This is a forecast, not an empirical claim about artificial superintelligence. Better cognition can reduce orchestration scaffolding; it does not remove differences in ownership, intent, access rights or evidence.

Test the UI with real comprehension tasks

Ask power users to identify the source of an instruction, explain which target receives a skill, distinguish retained knowledge from current context, identify a winning override, and locate the snapshot behind a run. Record accuracy, time, navigation backtracking and confidence—not just preference.

Useful research, with limits

AGDebugger studied interactive debugging and steering of multi-agent systems, informed by five formative interviews and a 14-participant evaluation. It supports investigating inspectable execution histories and targeted intervention. It does not establish that a read-only configuration viewer improves comprehension; that remains a product-specific hypothesis. [15]

Recommended direction

Build a legible map of the setup with trustworthy drill-down. Let the agent continue editing canonical configuration. Treat a workflow canvas as an optional view of real orchestration, and runtime evidence as an optional extension—not the definition of the product.

Official sources and research

Sources accessed 13 October 2026. Platform descriptions reflect official documentation. Design recommendations are analytical conclusions, not measured usability results.

[1] Dify · Build an Agent (new Agent beta)

https://docs.dify.ai/en/cloud/use-dify/build/new-agent/build

[2] Letta · MemFS

https://docs.letta.com/concepts/memfs

[3] LangChain · Introducing LangSmith Fleet

https://www.langchain.com/blog/introducing-langsmith-fleet

[4] LangChain · How to Use Memory in Agent Builder

https://www.langchain.com/blog/how-to-use-memory-in-agent-builder

[5] CrewAI · Crew Studio

https://docs-platform.crewai.com/platform/en/features/crew-studio

[6] CrewAI · Agent Control Plane policies

https://docs-platform.crewai.com/platform/en/features/agent-control-plane/policies

[7] Relevance AI · Workforces

https://relevanceai.com/docs/get-started/core-concepts/workforces

[8] Flowise · Agentflow V2

https://docs.flowiseai.com/using-flowise/agentflowv2

Official sources and research

[9] Langflow · Components overview

https://docs.langflow.org/concepts-components

[10] Google Cloud · Design agents in Agent Studio

https://docs.cloud.google.com/gemini-enterprise-agent-platform/agent-studio/design-agents?hl=en

[11] Microsoft · Agent development lifecycle in Foundry

https://learn.microsoft.com/en-us/azure/foundry/agents/concepts/development-lifecycle

[12] AWS · View information about versions of agents

https://docs.aws.amazon.com/bedrock/latest/userguide/agents-version-view.html

[13] Microsoft · AutoGen repository lifecycle notice

https://github.com/microsoft/autogen/blob/main/README.md

[14] Microsoft · AutoGen Studio usage

https://microsoft.github.io/autogen/stable/user-guide/autogenstudio-user-guide/usage.html

[15] Epperson et al. · Interactive Debugging and Steering of Multi-Agent AI Systems (2025)

https://arxiv.org/abs/2503.02068

Currency and scope

Dify’s new Agent is beta. Fleet is the renamed Agent Builder, distinct from LangSmith Studio. Letta’s legacy three-pane ADE should not be treated as its current interface. AutoGen is a historical UI reference. Vendor documentation can change; omissions in this review are not evidence of absent features.